EX6. Algorithm Selection & Results.

Our group decided to implement the following evolutionary algorithms, with our limited variation operators. We believe these are the best combinations:

1. Edge Recombination Crossover (ERX) + Inversion Mutation + Tournament Selection
2. Partially Mapped Crossover (PMX) + Swap Mutation + Elitism
3. Order Crossover (OX) + Insert Mutation + Fitness-Proportional Selection + Elitism

Justification

Edge Recombination Crossover (ERX) Algorithm (alg_erx.py)

ERX helps to preserve common edges in the population. This can provide significant improvements when paired with survivor selection, filtering out suboptimal city pairings and exploiting common beneficial pairings.

Tournament selection helps to maintain diversity in earlier iteration, preventing early convergence while enabling ERX to be more exploitative later on.

Inversion mutation helps exploration by disrupting the order of ERX, while retaining adjacencies between elements. It fine-tunes solution subpaths, balancing exploration and exploration.

Partially Mapped Crossover (PMX) Algorithm (alg_pmx.py)

Elitism is a very exploitative selection algorithm. **PMX** helps to preserve beneficial sequences found from Elitism, and provide significant improvements in a few iterations. Swap mutation ensures some exploration, while retaining sequence to be as exploitative as possible..

Order Crossover (OX) Algorithm (alg_ox.py)

Fitness-proportional selection provides a balance of exploration and exploitation. This was paired with **OX crossover** and **insert mutation**. OX maintains beneficial orders in each parent, which could be fine-tuned by insertion mutation. Elitism ensures that the best solutions are retained for future generations which was an issue that was encountered whereby initial solutions were often better than final solutions. This resulted in significantly more cost-efficient network traversals with greater consistency.

Conclusion

PMX gave the best results compared to OX. Its exploitative nature means that as population size and iteration increase, it will lead to much more efficient and favorable results. Although OX is much more explorative, the results stagnated much quicker as iteration and population size increased. Unfortunately, we could not run our ERX evolutionary algorithm.

Here are the results for Experiment 2, which will also be included in pmx_P50_I20K_results:

['eil51', '50', '20000']

Q1:600.2627708030063, med:602.0249513673793, q3:604.405687178264

['eil76', '50', '20000']

Q1:1068.5907695627905, med:1076.4345526073444, q3:1076.5492712956802

['eil101', '50', '20000']

Q1:1564.7555997947054, med:1568.4518496526732, q3:1568.860674107015

['st70', '50', '20000']

q1:1381.297566281885, med:1383.9162276397306, q3:1383.9162276397306

['kroA100', '50', '20000']

Q1:68788.99602955836, med:68797.05363356431, q3:68816.8052351981

['kroC100', '50', '20000']

Q1:65777.04874221655, med:66022.93340585304, q3:66159.30845036026

['lin105', '50', '20000']

Q1:44869.502521295144, med:44869.502521295144, q3:44893.509441401045

['pcb442', '50', '20000']

Q1:486813.9247372182, med:487349.33041347476, q3:487532.6083273021

['pr2392', '50', '20000']

Q1:12641530.997768685, med:12647711.961726563, q3:12650476.164625775

['usa13509', '50', '20000']

Q1:2070597273.7741385, med:2070733212.9371364, q3:2070828491.5091314